

Práctica 5

Capa de Transporte - Parte 1

Introducción

1. ¿Cuál es la función de la capa de transporte?

Esta capa es el "motor" encargado de garantizar que los datos viajen de manera segura, eficiente y ordenada desde el dispositivo de origen hasta el de destino.

La función principal de la capa de transporte es **proporcionar una comunicación lógica entre procesos de aplicación que se ejecutan en hosts diferentes.**

Desde la perspectiva de las aplicaciones, esta capa crea la ilusión de que los hosts están conectados directamente, incluso si están en lados opuestos del planeta y separados por numerosos enrutadores y enlaces físicos.

Mientras que la capa de red proporciona comunicación lógica entre los hosts (los dispositivos en sí), la capa de transporte extiende ese servicio de host a host para lograr una entrega de datos de proceso a proceso (sirviendo a la capa de aplicación), una función fundamental que se conoce como multiplexación y demultiplexación de la capa de transporte

2. Describa la estructura del segmento TCP y UDP.

Un segmento es el término utilizado para referirse a un paquete de datos específico de la capa de transporte de una red. Se construye cuando esta capa toma un mensaje proveniente de la capa de aplicación y lo encapsula añadiéndole información de cabecera propia, como los puertos de origen y destino y datos para la detección de errores.

UDP:

La estructura del segmento UDP (definida en el RFC 768) se caracteriza por ser sencilla y ligera, consistiendo en un encabezado de solo 8 bytes (formado por cuatro campos de dos bytes cada uno) seguido por el campo de datos

Sus campos principales son:

- Puertos de origen y destino: Permiten al host de destino entregar los datos de la aplicación al proceso correcto, llevando a cabo la función de demultiplexación
- Longitud (Length): Especifica el número total de bytes en el segmento UDP, abarcando tanto el encabezado como los datos. Este campo es explícito y necesario porque el tamaño de los datos puede variar de un segmento a otro
- Checksum: Es utilizada por el host receptor para verificar si se han introducido errores (bits alterados) en el segmento a medida que este viaja desde el origen hasta su destino
- Datos de aplicación (Payload): Ocupa el campo de datos y contiene la información que transfiere la aplicación subyacente (por ejemplo, consultas DNS o muestras de audio para streaming)

TCP:

La estructura del segmento TCP es considerablemente más robusta y compleja, ya que debe incluir la información de estado necesaria para proporcionar su servicio de transferencia de datos confiable.

Consiste en un campo de datos cuyo tamaño máximo está dictado por el Tamaño Máximo de Segmento (MSS) y un encabezado que típicamente tiene un tamaño fijo de 20 bytes (12 bytes más que UDP), aunque puede variar.

Los componentes de su encabezado son:

- **Puertos de origen y destino y Checksum:** Al igual que en UDP, cumplen funciones de multiplexación/demultiplexación hacia las aplicaciones y de detección de errores, respectivamente
- **Sequence y Acknowledgment number (32 bits cada uno):**
Son dos de los campos más importantes para garantizar un servicio de transferencia confiable. El número de secuencia refleja la posición del primer byte de datos de ese segmento en el flujo general de bytes de la conexión
- **Window size (16 bits):** Se utiliza para el mecanismo de control de flujo, indicando el número de bytes de espacio libre en el búfer que el receptor está dispuesto a aceptar sin saturarse
- **Longitud del encabezado (Header length , 4 bits):** Especifica la longitud total del encabezado TCP en palabras de 32 bits. Es fundamental ya que el tamaño del encabezado puede ser variable si se incluyen opciones
- **Flags, 6 bits:** Contiene bits de control para gestionar la conexión
 - El bit ACK indica que el valor llevado en el campo de acuse de recibo es válido
 - Los bits RST, SYN y FIN se usan para iniciar y cerrar la conexión TCP
 - Los bits CWR y ECE sirven para las notificaciones explícitas de congestión provenientes de la red
 - El bit PSH indica al receptor que los datos deben pasarse inmediatamente a la capa superior
 - El bit URG señala que el segmento contiene datos marcados como "urgentes" por la aplicación
- **Urgent pointer (16 bits):** Cuando el bit URG está activo, este campo indica la ubicación exacta del último byte de los datos urgentes dentro del segmento
- **Opciones (longitud variable y opcional):** Se emplea cuando el emisor y el receptor negocian el Tamaño Máximo de Segmento (MSS) o un factor de escala de ventana para su uso en redes de alta velocidad, además de incluir un sello de tiempo (timestamping)

3. ¿Cuál es el objetivo del uso de puertos en el modelo TCP/IP?

Un puerto es una interfaz lógica (o de software) que funciona como un punto de acceso específico en un dispositivo.

El objetivo principal del uso de puertos es llevar a cabo la multiplexación y demultiplexación en la capa de transporte. Dado que un host puede estar ejecutando simultáneamente muchas aplicaciones de red diferentes, la dirección IP del host no basta por sí sola para determinar a dónde debe ir la información.

Los números de puerto permiten al sistema operativo identificar de manera exacta a qué proceso de aplicación receptor (o remitente) pertenece un mensaje.

Cuando un paquete llega a su destino, la capa de transporte examina el número de puerto de destino en el encabezado del segmento y dirige (demultiplexa) los datos directamente al socket correcto de la aplicación correspondiente.

A nivel técnico, un puerto se representa mediante un número de 16 bits, lo que significa que existen 65536 puertos posibles (del 0 al 65535)

4. Compare TCP y UDP en cuanto a:

a. Confiabilidad

- **TCP:**

Tiene alta confiabilidad. Garantiza que todos los datos lleguen sin errores, completos y en orden mediante retransmisiones.

- **UDP:**

No confiable. Envía los datos sin verificar si llegaron, si se perdieron o si se duplicaron.

b. Multiplexación.

- **TCP:**

Basada en sockets completos: [IP origen, Puerto origen, IP destino, Puerto destino]. Esto permite que el receptor multiplexe múltiples conexiones simultáneas orientadas a un mismo puerto local, derivando cada flujo a un socket exclusivo e independiente por cliente.

- **UDP:**

Basada únicamente en el puerto de destino (y la IP de destino). El receptor recibe todos los datagramas entrantes destinados a ese puerto a través de una única "bandeja de entrada" común, sin importar el origen. La separación de respuestas se debe hacer analizando las cabeceras de cada datagrama de forma individual.

c. Orientado a la conexión.

- **TCP:**

Sí. Requiere el establecimiento previo de un canal lógico y bidireccional mediante el acuerdo de tres vías (Three-way handshake) para sincronizar números de secuencia antes de transferir datos. Al finalizar, requiere un cierre formal.

- **UDP:**

No, no está el concepto de conexión.

UDP envía los datos directamente de forma independiente (datagramas) sin aviso previo.

d. Controles de congestión.

- **TCP:**

Sí. Cuenta con mecanismos dinámicos (como Arranque Lento / Slow Start y Evitación de Congestión) que analizan la pérdida de paquetes y los tiempos de ida y vuelta (RTT) para regular la velocidad de inyección de datos, protegiendo la capacidad de transporte de la red general

- **UDP:**

No. No analiza el estado de la red; envía los datos a la máxima velocidad que le permite la aplicación.

e. Utilización de puertos.

- **TCP:**

Utiliza un espacio de direccionamiento de puertos de 16 bits (0 al 65535) independiente. Los puertos se asocian a un estado de conexión específico (ej. LISTEN, ESTABLISHED).

- **UDP:**

Utiliza su propio espacio de direccionamiento de puertos de 16 bits (0 al 65535). Al no manejar estados, los puertos actúan meramente como puntos de acceso o buzones de entrega inmediata. No interfieren ni se pisan con los puertos TCP, permitiendo que conviva un mismo número de puerto para protocolos distintos.

5. La PDU de la capa de transporte es el segmento. Sin embargo, en algunos contextos suele utilizarse el término datagrama. Indique cuando.

El uso de estos términos depende estrictamente de qué protocolo de la capa de transporte se esté utilizando y de si el servicio es orientado a la conexión o no.

Se utiliza el término Segmento cuando se trabaja con el protocolo TCP (o cualquier protocolo orientado a la conexión). Como requiere un control estricto de orden, retransmisiones y ventanas de flujo, cada unidad de datos está íntimamente ligada a la anterior y a la siguiente dentro de una conexión establecida.

Se utiliza el término Datagrama (específicamente Datagrama de Usuario) cuando se trabaja con el protocolo UDP (no orientado a la conexión). En UDP, cada unidad de datos se envía de manera totalmente independiente, como si fuera una carta postal física. No hay un "flujo" continuo ni relación entre un paquete y otro; cada bloque viaja con toda la información de direccionamiento necesaria para llegar por sí mismo (o perderse) sin previo aviso ni confirmación

6. Describa el saludo de tres vías de TCP. ¿UDP tiene esta característica?

El *Three-Way Handshake* es el proceso ordenado que utiliza TCP para establecer una conexión confiable y bidireccional entre un cliente y un servidor antes de transmitir datos reales.

Su objetivo principal es sincronizar los números de secuencia iniciales de ambos dispositivos y confirmar que ambos extremos están listos para comunicarse.

Como dice su nombre, consta de 3 pasos

- **Paso 1: SYN (Sincronización):**
 - El cliente inicia el proceso enviando un segmento especial al servidor con el flag SYN activado (en 1).
 - En este segmento, el cliente genera un número de secuencia inicial aleatorio (ej. X).
 - Con esto le dice al servidor: *"Quiero iniciar una conexión con vos y mi flujo de bytes va a empezar en el número X"*.
 - El cliente queda en estado **SYN-SENT**.
- **Paso 2: SYN-ACK (Sincronización y Acuse de recibo)**
 - El servidor recibe el requerimiento. Si tiene el puerto abierto y acepta la conexión, responde con un segmento que tiene activos los flags **SYN** y **ACK** al mismo tiempo.
 - **ACK:** Confirma que recibió el segmento del cliente haciendo $X + 1$ (indica el próximo byte que espera recibir).
 - **SYN:** El servidor también genera su propio número de secuencia inicial aleatorio (ej. Y) para su canal de envío.Con esto dice: *"Recibí tu pedido (ACK). Yo también quiero sincronizarme con vos y mi flujo va a empezar en el número Y (SYN)"*.
 - El servidor queda en estado **SYN-RECEIVED**.
- **Paso 3: ACK (Acuse de recibo)**
 - Finalmente, el cliente recibe la respuesta del servidor y envía un último segmento con el flag ACK activado.
 - **ACK:** Confirma la recepción del SYN del servidor haciendo $Y + 1$. Con esto dice: *"Perfecto, recibí tu número de secuencia y estoy listo"*.
 - A partir de este instante, ambos dispositivos cambian al estado **ESTABLISHED** y la conexión queda formalmente abierta para transmitir los datos de la aplicación.

¿UDP tiene esta característica?

No, en absoluto. UDP no tiene saludo de tres vías ni ningún mecanismo equivalente.

Como UDP es un protocolo no orientado a la conexión, carece por completo de la fase de establecimiento o negociación inicial. Si una aplicación usa UDP, simplemente empaqueta los datos en un datagrama y los inyecta en la red hacia la IP y puerto de destino, sin importar si el receptor está encendido, si el puerto está escuchando o si la red está en condiciones de procesarlo.

Por esta razón, UDP es muchísimo más rápido al iniciar la transmisión (ahorra el tiempo de ida y vuelta o RTT del saludo), pero traslada toda la responsabilidad de control de errores y caídas a la aplicación.

7. Investigue qué es el ISN (Initial Sequence Number). Relaciónelo con el saludo de tres vías.

El ISN (Initial Sequence Number) es el número de secuencia inicial que cada dispositivo (cliente y servidor) asigna al primer byte de datos que va a enviar al establecer una conexión TCP. El ISN es un valor de 32 bits generado de forma pseudoaleatoria por el sistema operativo de cada máquina.

El ISN es el núcleo del saludo de tres vías. No podés tener uno sin el otro, ya que la fase de "Sincronización" (SYN) sirve específicamente para que ambas partes se informen mutuamente cuál va a ser su ISN. Luego, cada byte de datos que se transmita en la conexión se irá sumando incrementalmente a partir de esos números base.

8. Investigue qué es el MSS. ¿Cuándo y cómo se negocia?

El MSS (Maximum Segment Size) es el tamaño máximo de datos (en bytes) que un dispositivo está dispuesto a aceptar en un único segmento TCP.

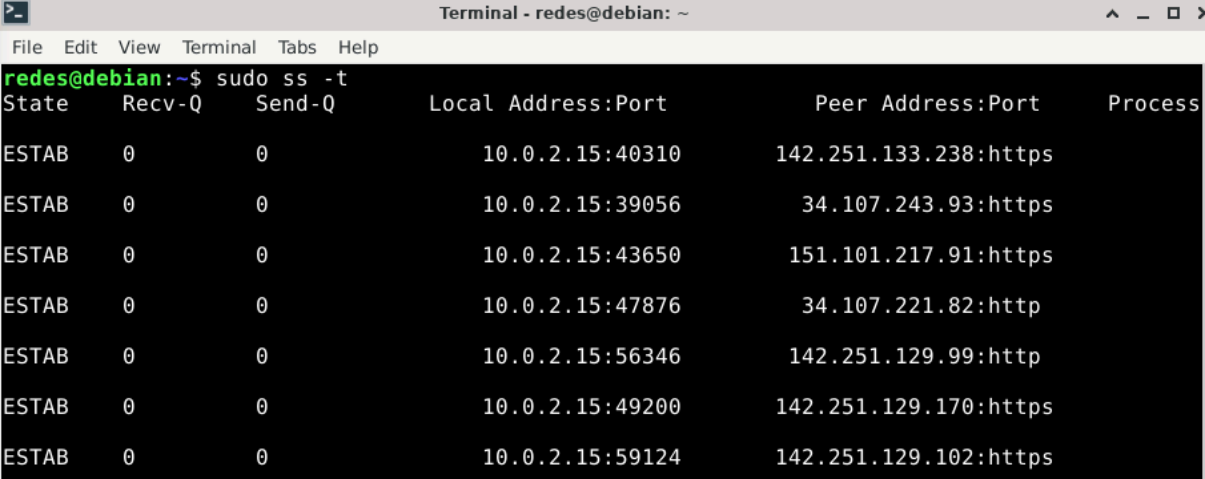
Se negocia únicamente durante el Three-Way Handshake, al establecerse la conexión. Después de esa fase, el valor no se puede modificar.

Se negocia a través del campo Options (Opciones) del encabezado TCP:

- El cliente envía su valor de MSS en el segmento SYN.
- El servidor envía su propio valor de MSS en el segmento SYN-ACK.
- **La regla:** Cada extremo toma el menor de los dos valores para asegurar que ninguno de los dos sature los búfers del otro ni provoque fragmentación IP en el camino. No es una negociación donde se "promedia", sino que se impone el límite más restrictivo.

9. Utilice el comando ss (reemplazo de netstat) para obtener la siguiente información de su PC:

a. Para listar las comunicaciones TCP establecidas.



```
Terminal - redes@debian: ~
File Edit View Terminal Tabs Help
redes@debian:~$ sudo ss -t
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
ESTAB	0	0	10.0.2.15:40310	142.251.133.238:https	
ESTAB	0	0	10.0.2.15:39056	34.107.243.93:https	
ESTAB	0	0	10.0.2.15:43650	151.101.217.91:https	
ESTAB	0	0	10.0.2.15:47876	34.107.221.82:http	
ESTAB	0	0	10.0.2.15:56346	142.251.129.99:http	
ESTAB	0	0	10.0.2.15:49200	142.251.129.170:https	
ESTAB	0	0	10.0.2.15:59124	142.251.129.102:https	

b. Para listar las comunicaciones UDP establecidas.

```
redes@debian:~$ ss -u
```

Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
0	0	10.0.2.15%enp0s3:bootpc	10.0.2.2:bootps	

Como UDP no tiene estados reales de conexión, esto te nos muestra solo los flujos UDP activos que estén enviando/recibiendo tráfico en ese momento

c. Obtener sólo los servicios TCP que están esperando comunicaciones

```
redes@debian:~$ ss -t -l
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	128	0.0.0.0:ssh	0.0.0.0:*	
LISTEN	0	128	127.0.0.1:ipp	0.0.0.0:*	
LISTEN	0	5	127.0.0.1:4038	0.0.0.0:*	
LISTEN	0	128	:::ssh	:::*	
LISTEN	0	128	:::1:ipp	:::*	
LISTEN	0	4096	:::1:50051	:::*	
LISTEN	0	4096	:::ffff:127.0.0.1:50051	:::*	

La bandera -l filtra únicamente los sockets en modo de escucha o LISTEN

d. Obtener sólo los servicios UDP que están esperando comunicaciones.

```
redes@debian:~$ ss -u -l
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
UNCONN	0	0	0.0.0.0:mdns	0.0.0.0:*	
UNCONN	0	0	0.0.0.0:40427	0.0.0.0:*	
UNCONN	0	0	0.0.0.0:631	0.0.0.0:*	
UNCONN	0	0	127.0.0.1:4038	0.0.0.0:*	
UNCONN	0	0	:::42137	:::*	
UNCONN	0	0	:::mdns	:::*	

e. Repetir los anteriores para visualizar el proceso del sistema asociado a la conexión. f. Obtenga la misma información planteada en los ítems anteriores usando el comando netstat.

```
Terminal - redes@debian: ~
File Edit View Terminal Tabs Help
redes@debian:~$ netstat -t | grep ESTABLISHED
tcp        0      0 10.0.2.15:39056      93.243.107.34.bc.:https ESTABLISHED
redes@debian:~$ netstat -u
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp        0      0 10.0.2.15:bootpc      10.0.2.2:bootps        ESTABLISHED
redes@debian:~$ netstat -t -l
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:ssh            0.0.0.0:*              LISTEN
tcp        0      0 localhost:ipp           0.0.0.0:*              LISTEN
tcp        0      0 localhost:4038          0.0.0.0:*              LISTEN
tcp6       0      0 [::]:ssh               [::]:*                 LISTEN
tcp6       0      0 localhost:ipp           [::]:*                 LISTEN
tcp6       0      0 localhost:50051         [::]:*                 LISTEN
tcp6       0      0 127.0.0.1:50051        [::]:*                 LISTEN
redes@debian:~$ netstat -u -l
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp        0      0 0.0.0.0:mdns           0.0.0.0:*              LISTEN
udp        0      0 0.0.0.0:40427          0.0.0.0:*              LISTEN
udp        0      0 0.0.0.0:631            0.0.0.0:*              LISTEN
udp        0      0 localhost:4038          0.0.0.0:*              LISTEN
udp6       0      0 [::]:42137             [::]:*                 LISTEN
udp6       0      0 [::]:mdns              [::]:*                 LISTEN
```

10. ¿Qué sucede si llega un segmento TCP con el flag SYN activo a un host que no tiene ningún proceso esperando en el puerto destino de dicho segmento (es decir, el puerto destino no está en estado LISTEN)?

Si un host recibe un segmento TCP con el flag SYN activo (un intento de iniciar una conexión) apuntando a un puerto donde no hay ninguna aplicación escuchando (es decir, el puerto no está en estado LISTEN), el sistema operativo del host de destino rechazará la conexión de inmediato.

El host de destino responderá enviando un segmento TCP de vuelta con el flag RST (Reset) y el flag ACK activados simultáneamente.

- **El flag RST:** Le indica de forma tajante al emisor que la conexión no se puede establecer y que debe abortar el intento inmediatamente, destruyendo cualquier estado temporal en su memoria.
- **El flag ACK:** Se incluye para confirmar la recepción del segmento SYN original. El número de acuse de recibo (ACK) se establece igual al número de secuencia del SYN entrante más uno (SYNseq + 1).

Al recibir este paquete RST-ACK, la computadora de origen sabe que el puerto está cerrado y, a nivel de aplicación, el programa cliente suele arrojar un error del tipo "Connection refused" (Conexión rechazada).

a. Utilice hping3 para enviar paquetes TCP al puerto destino 22 de la máquina virtual con el flag SYN activado.

```
redes@debian:~$ sudo hping3 -S -p 22 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): S set, 40 headers + 0 data bytes
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=0 win=65495 rtt=3.6 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=1 win=65495 rtt=6.9 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=2 win=65495 rtt=1.0 ms

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 1.0/3.9/6.9 ms
```

b. Utilice hping3 para enviar paquetes TCP al puerto destino 40 de la máquina virtual con el flag SYN activado.

```
redes@debian:~$ sudo hping3 -S -p 40 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): S set, 40 headers + 0 data bytes
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=0 win=0 rtt=8.1 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=1 win=0 rtt=12.0 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=2 win=0 rtt=3.9 ms

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 3.9/8.0/12.0 ms
```

c. ¿Qué diferencias nota en las respuestas obtenidas en los dos casos anteriores? ¿Puede explicar a qué se debe? (Ayuda: utilice el comando ss visto anteriormente).

La diferencia radical entre ambas respuestas radica en el estado de escucha de los puertos en el host de destino, lo cual determina el comportamiento del protocolo TCP:

- Puerto 22 (SYN-ACK): Al recibir el segmento con el flag SYN activo, el sistema operativo acepta la solicitud de inicio de conexión debido a que el servicio SSH se encuentra corriendo y el puerto está en estado LISTEN. Por lo tanto, responde con el segundo paso Three-way handshake), enviando flags SYN-ACK.
- Puerto 40 (RST-ACK): Al recibir el segmento SYN en el puerto 40, el sistema operativo comprueba que no existe ningún proceso o servicio esperando conexiones en dicho puerto (no está en estado LISTEN). Siguiendo el estándar del protocolo, el host rechaza la conexión de forma tajante enviando un paquete de reinicio RST (Reset) junto con un ACK para indicarle al emisor que aborte el intento inmediatamente.

11. ¿Qué sucede si llega un datagrama UDP a un host que no tiene ningún proceso esperando en el puerto destino de dicho datagrama (es decir, que dicho puerto no está en estado LISTEN)?

Cuando un host recibe un datagrama UDP en un puerto donde no hay ninguna aplicación escuchando, el protocolo UDP no puede responder por sí mismo (porque no tiene flags como RST).

En su lugar, la capa de transporte le avisa a la capa de red, y el sistema operativo del host de destino genera y envía de vuelta un paquete ICMP (tipo 3, código 3), que significa "Puerto inaccesible" (Port Unreachable).

a. Utilice hping3 para enviar datagramas UDP al puerto destino 5353 de la máquina virtual.

```
redes@debian:~$ sudo hping3 --udp -p 5353 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): udp mode set, 28 headers + 0 data bytes

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

b. Utilice hping3 para enviar datagramas UDP al puerto destino 40 de la máquina virtual.

```
redes@debian:~$ sudo hping3 --udp -p 40 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): udp mode set, 28 headers + 0 data bytes
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=1595 seq=0
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=1596 seq=1
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=1597 seq=2

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 6.7/15.0/25.2 ms
```

c. ¿Qué diferencias nota en las respuestas obtenidas en los dos casos anteriores? ¿Puede explicar a qué se debe? (Ayuda: utilice el comando ss visto anteriormente).

Puerto 5353 (Abierto): Se observa que el Host no nos respondió. Cuando el puerto está abierto, el protocolo UDP simplemente entrega los datos a la aplicación en silencio. Al no existir un mecanismo de avisar que le llegaron las cosas en UDP (porque no hay una conexión), no vuelve ningún paquete al emisor.

Puerto 40 (Cerrado): Se obtuvo un 0% de pérdida de paquetes, ya que el host respondió inmediatamente a cada datagrama con un mensaje de error: ICMP Port Unreachable (Puerto inaccesible).

A diferencia de TCP, el protocolo UDP no está orientado a la conexión y carece de banderas de control (como RST) para indicar errores. Por lo tanto:

- Cuando el puerto está cerrado (como el 40), el datagrama llega al sistema operativo de destino, pero al no encontrar ninguna aplicación asociada, la pila de red del kernel descarta el datagrama e interviene el protocolo ICMP (Capa de Red) para notificar el fallo de transporte al emisor mediante un mensaje de tipo 3, código 3 (Destination Unreachable - Port Unreachable).

12. Investigue los distintos tipos de estado que puede tener una conexión TCP.

Ver https://users.cs.northwestern.edu/~agupta/cs340/project2/TCPIP_State_Transition_Diagram.pdf

1. Fase de Establecimiento (Saludo de 3 vías)

- **CLOSED:** Es el punto de partida (ficticio) antes de que se inicie cualquier conexión. No existe ninguna estructura de control activa.
- **LISTEN:** El puerto está abierto del lado del servidor (apertura pasiva). Se queda esperando a que llegue una solicitud de conexión (SYN) de algún cliente remoto.
- **SYN-SENT:** El cliente inicia una apertura activa enviando un segmento **SYN** y se queda esperando el acuse de recibo correspondiente.
- **SYN-RECEIVED:** El servidor recibió el SYN del cliente, le respondió con un SYN-ACK, y ahora espera el último ACK de confirmación del cliente para asegurar que ambos se escuchan.

2. Fase de Transferencia de Datos

- **ESTABLISHED:** ¡Conexión totalmente abierta! Es el estado operativo normal. El Threeway Handshake finalizó con éxito y ambas partes pueden intercambiar datos de forma bidireccional de manera segura.

3. Fase de Cierre (Terminación de la Conexión)

Dado que TCP es *Full-Duplex*, ambos extremos deben cerrar su canal de forma independiente. Los estados cambian según quién tome la iniciativa:

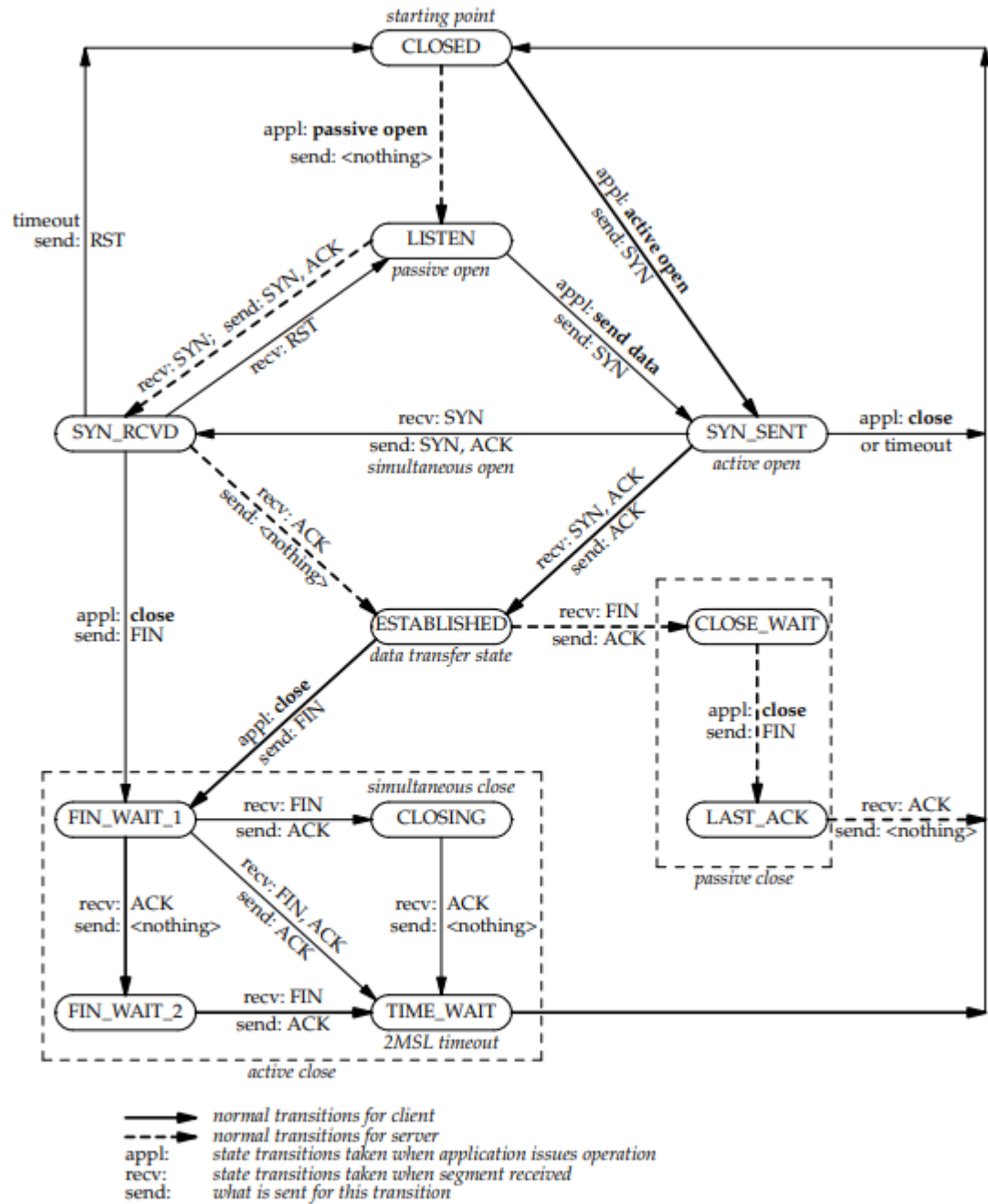
A. Lado que inicia el cierre (Cierre Activo):

- **FIN-WAIT-1:** La aplicación local ordena cerrar la conexión, envía un segmento **FIN** al destino y queda esperando que le confirmen el recibo con un **ACK**.
- **FIN-WAIT-2:** El host local ya recibió el **ACK** de su propio **FIN**. Ahora espera pasivamente a que el host remoto termine de enviar sus datos pendientes y mande su propio **FIN**.
- **CLOSING:** Un estado de transición inusual en el que ambas partes decidieron colgar al mismo tiempo (cierre simultáneo). Se envió un **FIN** y se recibió un **FIN**, pero aún falta recibir el **ACK** final.
- **TIME-WAIT:** El host local recibió el **FIN** del remoto y envió el último **ACK**. Entra en una espera de tiempo equivalente a 2 MSL (Máxima vida útil de un segmento) antes de pasar a **CLOSED**.
 - ¿Para qué sirve? Para garantizar que el último **ACK** llegue a destino (si se pierde, se retransmite) y para evitar que paquetes rezagados de esta conexión interfieran en conexiones futuras.

B. Lado que recibe la petición de cierre (Cierre Pasivo):

CLOSE-WAIT: El host remoto envió un **FIN** (avisando que no mandará más datos) y la máquina local le devolvió un **ACK**. El sistema operativo local se queda esperando en este estado hasta que la aplicación local termine sus procesos y dé la orden explícita de cerrar su extremo.

LAST-ACK: La aplicación local finalmente ordena el cierre, el sistema operativo envía su paquete **FIN** definitivo hacia el exterior y se queda esperando el último **ACK** remoto para dar por muerta la conexión y regresar a **CLOSED**.



TCP state transition diagram.

13. Dada la siguiente salida del comando ss, responde:

```
Netid State      Recv-Q Send-Q Local Address:Port      Peer Address:Port      Process
tcp    LISTEN         0      128      *:22                  *:*                      users:((("sshd",pid=468,fd=29))
tcp    LISTEN         0      128      *:80                  *:*                      users:((("apache2",pid=991,fd=95))
udp    LISTEN         0      128      163.10.5.222:53      *:*                      users:((("named",pid=452,fd=10))
tcp    ESTAB          0      0      163.10.5.222:59736    64.233.163.120:443      users:((("x-www-browser",pid=1079,fd=51))
tcp    CLOSE-WAIT     0      0      163.10.5.222:41654    200.115.89.30:443      users:((("x-www-browser",pid=1079,fd=50))
tcp    ESTAB          0      0      163.10.5.222:59737    64.233.163.120:443      users:((("x-www-browser",pid=1079,fd=55))
tcp    ESTAB          0      0      163.10.5.222:33583    200.115.89.15:443      users:((("x-www-browser",pid=1079,fd=53))
tcp    ESTAB          0      0      163.10.5.222:45293    64.233.190.99:443      users:((("x-www-browser",pid=1079,fd=59))
tcp    LISTEN         0      128      *:25                  *:*                      users:((("postfix",pid=627,fd=3))
tcp    ESTAB          0      0      127.0.0.1:22          127.0.0.1:41220        users:((("sshd",pid=1418,fd=3),
                                ("sshd",pid=1416,fd=3))
tcp    ESTAB          0      0      163.10.5.222:52952    64.233.190.94:443      users:((("x-www-browser",pid=1079,fd=29))
tcp    TIME-WAIT      0      0      163.10.5.222:36676    54.149.207.17:443      users:((("x-www-browser",pid=1079,fd=3))
tcp    ESTAB          0      0      163.10.5.222:52960    64.233.190.94:443      users:((("x-www-browser",pid=1079,fd=67))
tcp    ESTAB          0      0      163.10.5.222:50521    200.115.89.57:443      users:((("x-www-browser",pid=1079,fd=69))
tcp    SYN-SENT       0      0      163.10.5.222:52132    43.232.2.2:9500        users:((("x-www-browser",pid=1079,fd=70))
tcp    ESTAB          0      0      127.0.0.1:41220      127.0.0.1:22           users:((("ssh",pid=1415,fd=3))
udp    LISTEN         0      128      127.0.0.1:53         *:*                      users:((("named",pid=452,fd=9))
```

a. ¿Cuántas conexiones hay establecidas?

Hay 9 conexiones establecidas.

Hay que contar cuántas filas tienen el estado ESTAB (abreviatura de ESTABLISHED) en la segunda columna.

b. ¿Cuántos puertos hay abiertos a la espera de posibles nuevas conexiones?

Hay 5 puertos abiertos en espera.

Hay que contar cuántas filas tienen el estado LISTEN en la segunda columna.

c. El cliente y el servidor de las comunicaciones HTTPS (puerto 443), ¿residen en la misma máquina?

No, residen en máquinas distintas.

Si miramos las líneas que tengan el puerto :443 (HTTPS), se ve que la Local Address (163.10.5.222) se está comunicando con IPs públicas externas como 64.233.163.120, 200.115.89.15, etc. Como las IPs locales y remotas son diferentes, el servidor HTTPS está afuera en internet.

d. El cliente y el servidor de la comunicación SSH (puerto 22), ¿residen en la misma máquina?

Sí, residen en la misma máquina.

Si miramos las conexiones ESTAB asociadas al puerto 22, se ven dos líneas idénticas pero espejadas, donde cambia el puerto pero la IP es 127.0.0.1 (local host)

e. Liste los nombres de todos los procesos asociados con cada comunicación. Indique para cada uno si se trata de un proceso cliente o uno servidor.

Como saber cual es cual:

El Servidor siempre está atado a algun puerto bien conocido de la materia (22, 80, 443, 53, 25). El Cliente siempre usa un puerto aleatorio alto (ej. 59736, 41220) para originar la salida.

Lista:

- **sshd**: Es Servidor (LISTEN puerto 22) y es Cliente (ESTAB puerto 22 y 41220)
- **apache2**: Servidor (LISTEN puerto 80)
- **named**: Servidor (LISTEN puerto 53)
- **postfix**: Servidor (LISTEN puerto 25)
- **x-www-broser**: Cliente (del lado de la dirección local, pegando a 433 o 80)

f. ¿Cuáles conexiones tuvieron el cierre iniciado por el host local y cuáles por el remoto?

Usando lo del punto 12

- Si el estado es TIME-WAIT, significa que el host local inició el cierre activo (mandó el primer FIN, recibió el ACK/FIN y ahora espera el timeout de seguridad).
- Si el estado es CLOSE-WAIT, significa que el host remoto inició el cierre (nos mandó un FIN, nuestra máquina le tiró un ACK y estamos esperando que nuestra aplicación local se digne a cerrar).

Respuesta:

- Iniciado por el host local: 1 conexión (la que está en estado TIME-WAIT con la IP 54.149.207.17:443).
- Iniciado por el host remoto: 1 conexión (la que está en estado CLOSE-WAIT con la IP 200.115.89.30:443).

g. ¿Cuántas conexiones están aún pendientes por establecerse?

Hay 1 conexión pendiente.

Hay que buscar estados intermedios del Threeway Handshake (SYN-SENT o SYN-RCVD).

En la lista hay una fila que tiene el estado SYN-SENT (conectándose a la IP 43.232.2.2:9500). Eso significa que el cliente local mandó el SYN pero todavía no recibió el SYN-ACK de respuesta; la conexión está en trámite, no se terminó de armar.

14. Dadas las salidas de los siguientes comandos ejecutados en el cliente y el servidor, responder:

a. ¿Qué segmentos llegaron y cuáles se están perdiendo en la red?

Considerando que el saludo normal es:

- Cliente envía SYN y pasa a estado SYN-SENT.
- Servidor recibe SYN, responde con SYN-ACK y pasa a estado SYN-RECV.
- Cliente recibe SYN-ACK, responde con ACK y pasa a estado ESTABLISHED.

Segmentos que llegaron:

- Llegó el primer segmento (SYN) desde el cliente hacia el servidor.

¿Cómo me doy cuenta?

- Porque el servidor está en estado SYN-RECV , lo que significa que recibió con éxito la solicitud de apertura (SYN) del cliente.
- El cliente, por su parte, está en SYN-SENT , confirmando que lo envió de manera activa.

Segmentos que se están perdiendo:

- Se está perdiendo el segundo segmento (SYN-ACK) que el servidor le envía de vuelta al cliente.

¿Cómo me doy cuenta?

- El servidor ya procesó el SYN y pasó a SYN-RECV, estado en el cual el protocolo TCP automáticamente despacha el SYN-ACK hacia el cliente.
- Sin embargo, el cliente se quedó "tildado" en SYN-SENT esperando.
- Si al cliente le hubiera llegado el SYN-ACK, ya habría avanzado al estado ESTAB. Como sigue esperando, significa que el SYN-ACK aún no le llegó.

b. ¿A qué protocolo de capa de aplicación y de transporte se está intentando conectar el cliente?

Protocolo de Capa de Transporte:

- TCP. La primera columna de la salida de ss lo dice explícitamente: tcp. Además, los estados SYN-SENT y SYN-RECV son exclusivos de TCP.

Protocolo de Capa de Aplicación:

- POP3 (Post Office Protocol v3). Nos lo indica el puerto de destino del servidor, que es el :110 (157.0.0.1:110). Por estándar de la IANA, el puerto 110 está reservado mundialmente para POP3, el protocolo de capa de aplicación que se usa para descargar correos electrónicos desde un servidor.

c. ¿Qué flags tendría seteado el segmento perdido?

El segmento perdido tendría seteados los flags SYN y ACK simultáneamente (un paquete SYN-ACK)

15. Use CORE para armar una topología como la siguiente, sobre la cual deberá realizar: a. En ambos equipos inspeccionar el estado de las conexiones y mantener abiertas ambas ventanas con el comando corriendo para poder visualizar los cambios a medida que se realiza el ejercicio. Ayuda: `watch -n1 'ss -nat'`. b. En Servidor, utilice la herramienta `ncat` para levantar un servicio que escuche en el puerto 8001/TCP. Utilice la opción `-k` para que el servicio sea persistente. Verifique el estado de las conexiones. c. Desde CLIENTE1 conectarse a dicho servicio utilizando también la herramienta `ncat`. Inspeccione el estado de las conexiones. d. Iniciar otra conexión desde CLIENTE1 de la misma manera que la anterior y verificar el estado de las conexiones. ¿De qué manera puede identificar cada conexión? e. En base a lo observado en el item anterior, ¿es posible iniciar más de una conexión desde el cliente al servidor en el mismo puerto destino? ¿Por qué? ¿Cómo se garantiza que los datos de una conexión no se mezclarán con los de la otra? f. Analice en el tráfico de red, los flags de los segmentos TCP que ocurren cuando: i. Cierra la última conexión establecida desde CLIENTE1. Evalúe los estados de las conexiones en ambos equipos. ii. Corta el servicio de `ncat` en el servidor (Ctrl+C). Evalúe los estados de las conexiones en ambos equipos. iii. Cierra la conexión en el cliente. Evalúe nuevamente los estados de las conexiones.



el tiempo es oro.